

HTML & CSS Complete Reference Guide

Tags, Attributes, Properties & Worked Examples for Teaching

Compiled July 2026

This guide covers HTML structure, common tags and their attributes, forms, tables, and media — followed by CSS selectors, the box model, layout systems (Flexbox & Grid), typography, colors, and responsive design. Each entry includes a plain-language description, a syntax pattern or attribute table, and a runnable example, making it suitable for classroom hand-outs, lab exercises, or self-study.

Contents

1. HTML Document Structure
2. Text & Content Tags
3. Links, Images & Media
4. Lists & Tables
5. Forms & Input Elements
6. Semantic Layout Tags
7. CSS Basics — Syntax & Selectors
8. The Box Model
9. Typography & Colors
10. Flexbox Layout
11. CSS Grid Layout
12. Positioning, Display & Responsive Design

1. HTML Document Structure

HTML (HyperText Markup Language) describes the structure of a web page using nested elements made of opening and closing tags: `<tag>content</tag>`. Every HTML document follows the same basic skeleton.

`<!DOCTYPE html>`

Declares the document as HTML5, telling the browser which rules to use when rendering the page. It must be the very first line of an HTML file.

EXAMPLE

```
<!DOCTYPE html>
<html>
  ...
</html>
```

`<html>`

The root element that wraps the entire page. The `lang` attribute declares the page's primary language for accessibility and search engines.

COMMON ATTRIBUTES

Attribute	Description
<code>lang</code>	Language code of the document, e.g. 'en', 'es', 'fr'

EXAMPLE

```
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

`<head>`

Contains metadata about the page — title, character set, linked stylesheets, and scripts — none of which is displayed directly on the page.

EXAMPLE

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
```

`<meta>`

Provides metadata such as character encoding, page description, or the responsive viewport setting. It is a self-closing/void element with no closing tag.

COMMON ATTRIBUTES

Attribute	Description
<code>charset</code>	Character encoding, typically UTF-8
<code>name</code>	Type of metadata, e.g. 'viewport', 'description', 'author'
<code>content</code>	The value associated with the name attribute

EXAMPLE

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="A page about HTML and CSS">
```

<title>

Sets the text shown in the browser tab and used as the default bookmark name.

EXAMPLE

```
<title>My First Web Page</title>
```

<link>

Connects an external resource, most commonly a CSS stylesheet, to the document. It is a void element placed inside <head>.

COMMON ATTRIBUTES

Attribute	Description
rel	Relationship to the document, e.g. 'stylesheet', 'icon'
href	Path or URL to the linked resource
type	MIME type of the linked resource (optional)

EXAMPLE

```
<link rel="stylesheet" href="style.css">
```

<body>

Contains all the visible content of the page — text, images, links, forms, and other elements.

EXAMPLE

```
<body>
  <h1>Welcome</h1>
  <p>This is my page.</p>
</body>
```

<script>

Embeds or links to JavaScript code. Placing it near the end of <body> lets the page content load first.

COMMON ATTRIBUTES

Attribute	Description
src	Path to an external JavaScript file
defer	Delays execution until the HTML is fully parsed
async	Loads and runs the script without blocking parsing

EXAMPLE

```
<script src="app.js"></script>
<script>
  console.log('Page loaded');
</script>
```

2. Text & Content Tags

These tags structure and emphasize the text content of a page.

<h1> – <h6>

Six levels of headings, from <h1> (most important) to <h6> (least). Use them in order to build a logical document outline rather than for font size alone.

EXAMPLE

```
<h1>Main Title</h1>
<h2>Section Heading</h2>
<h3>Subsection Heading</h3>
```

<p>

Defines a paragraph of text, automatically adding space before and after it.

EXAMPLE

```
<p>This is a paragraph of text describing the topic.</p>
```


An inline container with no default styling, used to apply a class, id, or style to a small piece of text within a larger block.

COMMON ATTRIBUTES

Attribute	Description
class	One or more CSS class names for styling
id	A unique identifier for the element

EXAMPLE

```
<p>The sky is <span class="highlight">blue</span> today.</p>
```

<div>

A generic block-level container used to group elements for styling or layout purposes.

COMMON ATTRIBUTES

Attribute	Description
class	One or more CSS class names for styling
id	A unique identifier for the element

EXAMPLE

```
<div class="card">
  <h3>Title</h3>
  <p>Card content.</p>
</div>
```

 /

 marks text as important (rendered bold); marks text as emphasized (rendered italic). Both carry semantic meaning, unlike /<i>.

EXAMPLE

```
<p><strong>Warning:</strong> this action <em>cannot</em> be undone.</p>
```


 / <hr>

 inserts a single line break within text. <hr> draws a horizontal rule to separate content sections. Both are void elements.

EXAMPLE

```
<p>Line one.<br>Line two.</p>
<hr>
```

<blockquote> / <q>

<blockquote> sets off a longer quoted block (often indented by the browser); <q> marks a short inline quotation.

COMMON ATTRIBUTES

Attribute	Description
cite	URL of the source being quoted

EXAMPLE

```
<blockquote cite="https://example.com">
  Quoted paragraph text goes here.
</blockquote>
```

<code> / <pre>

<code> marks inline code in a monospace font. <pre> preserves whitespace and line breaks exactly as written, commonly wrapping a <code> block.

EXAMPLE

```
<pre><code>function greet() {
  console.log('Hi');
}</code></pre>
```

3. Links, Images & Media

<a>

Creates a hyperlink to another page, file, email address, or location within the same page.

COMMON ATTRIBUTES

Attribute	Description
href	Destination URL, file path, or #anchor
target	'_blank' opens the link in a new tab
rel	Relationship, e.g. 'noopener noreferrer' with target='_blank'

EXAMPLE

```
<a href="https://example.com" target="_blank">Visit Example</a>
```


Embeds an image in the page. It is a void element, and the alt attribute is required for accessibility and displays if the image fails to load.

COMMON ATTRIBUTES

Attribute	Description
src	Path or URL of the image file
alt	Alternative text describing the image
width / height	Display dimensions in pixels
loading	'lazy' defers loading until near the viewport

EXAMPLE

```

```

<video>

Embeds video content with optional built-in playback controls.

COMMON ATTRIBUTES

Attribute	Description
src	Path or URL of the video file
controls	Shows play/pause/volume controls
autoplay	Starts playback automatically
loop	Replays the video when it ends

EXAMPLE

```
<video src="movie.mp4" controls width="640">  
  Your browser does not support video.  
</video>
```

<audio>

Embeds sound content with optional playback controls.

COMMON ATTRIBUTES

Attribute	Description
src	Path or URL of the audio file
controls	Shows play/pause/volume controls

EXAMPLE

```
<audio src="song.mp3" controls></audio>
```

4. Lists & Tables

 / /

 creates an unordered (bulleted) list, an ordered (numbered) list; each item is wrapped in .

EXAMPLE

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
</ul>

<ol>
  <li>Preheat oven</li>
  <li>Mix ingredients</li>
</ol>
```

<table> / <tr> / <td> / <th>

<table> defines a table; <tr> a row; <td> a standard data cell; <th> a header cell (rendered bold and centered by default).

COMMON ATTRIBUTES

Attribute	Description
colspan	Number of columns a cell should span
rowspan	Number of rows a cell should span

EXAMPLE

```
<table>
  <tr>
    <th>Name</th>
    <th>Score</th>
  </tr>
  <tr>
    <td>Ana</td>
    <td>92</td>
  </tr>
</table>
```

<thead> / <tbody> / <tfoot>

Groups table rows into a header, body, and footer section, improving structure and allowing independent styling of each part.

EXAMPLE

```
<table>
  <thead><tr><th>Item</th><th>Price</th></tr></thead>
  <tbody><tr><td>Pen</td><td>$1.20</td></tr></tbody>
  <tfoot><tr><td>Total</td><td>$1.20</td></tr></tfoot>
</table>
```

5. Forms & Input Elements

Forms collect user input and submit it to a server for processing.

<form>

Wraps a group of input controls and defines where and how their data is submitted.

COMMON ATTRIBUTES

Attribute	Description
action	URL that receives the submitted data
method	HTTP method used to submit, 'GET' or 'POST'

EXAMPLE

```
<form action="/submit" method="POST">
  ...
</form>
```

<input>

A versatile void element that renders differently based on its type attribute — text box, checkbox, radio button, date picker, and more.

COMMON ATTRIBUTES

Attribute	Description
type	Kind of control: text, email, password, checkbox, radio, date, number, etc.
name	Key used when the form data is submitted
placeholder	Hint text shown when the field is empty
required	Prevents submission until a value is provided
value	Default or current value of the field

EXAMPLE

```
<input type="text" name="username" placeholder="Enter your name" required>
<input type="email" name="email" required>
<input type="checkbox" name="subscribe" checked>
```

<label>

Attaches a readable caption to a form control, improving usability and accessibility. Clicking the label focuses the associated control.

COMMON ATTRIBUTES

Attribute	Description
for	Matches the id of the associated input element

EXAMPLE

```
<label for="email">Email address</label>
<input type="email" id="email" name="email">
```

<textarea>

A multi-line text input field for longer content such as comments or messages.

COMMON ATTRIBUTES

Attribute	Description
rows / cols	Visible height and width in text lines/characters
placeholder	Hint text shown when empty

EXAMPLE

```
<textarea name="message" rows="4" cols="40">Default text</textarea>
```

<select> / <option>

<select> creates a drop-down list; each choice is defined with an <option> tag.

COMMON ATTRIBUTES

Attribute	Description
value	Data sent when that option is chosen
selected	Marks an option as chosen by default

EXAMPLE

```
<select name="country">
  <option value="us">United States</option>
  <option value="ph" selected>Philippines</option>
</select>
```

<button>

A clickable button, most often used to submit or reset a form or trigger JavaScript.

COMMON ATTRIBUTES

Attribute	Description
type	'submit', 'reset', or 'button'

EXAMPLE

```
<button type="submit">Send</button>
<button type="button" onclick="doSomething()">Click Me</button>
```

6. Semantic Layout Tags

Semantic tags describe the meaning of a page section, improving accessibility and SEO compared to generic `<div>` elements.

`<header>`

Introductory content or navigation for a page or section, often containing a logo and nav.

`<nav>`

A block of primary navigation links.

`<main>`

The single, primary content area of the page, excluding repeated header/footer/nav content.

`<section>`

A thematic grouping of content, typically with its own heading.

`<article>`

Self-contained content that could stand alone, such as a blog post or news story.

`<aside>`

Content tangentially related to the main content, such as a sidebar.

`<footer>`

Closing content for a page or section, such as copyright or contact links.

EXAMPLE

```
<body>
  <header><h1>My Site</h1><nav>...</nav></header>
  <main>
    <article>
      <h2>Post Title</h2>
      <p>Post content...</p>
    </article>
    <aside>Related links</aside>
  </main>
  <footer>&copy; 2026 My Site</footer>
</body>
```

7. CSS Basics — Syntax & Selectors

CSS (Cascading Style Sheets) controls the visual presentation of HTML. A rule consists of a selector and a declaration block of property: value pairs.

Basic Rule Syntax

A selector targets one or more HTML elements; the declaration block sets their styles.

SYNTAX

```
selector {  
  property: value;  
  property: value;  
}
```

EXAMPLE

```
p {  
  color: #333333;  
  font-size: 16px;  
}
```

Linking CSS to HTML

CSS can be external (a separate .css file linked in <head>), internal (inside a <style> tag), or inline (a style attribute on one element). External stylesheets are preferred for maintainability.

SYNTAX

```
<link rel="stylesheet" href="style.css">  
  
<style> /* internal */ </style>  
  
<p style="color:red;"> <!-- inline --> </p>
```

EXAMPLE

```
<link rel="stylesheet" href="style.css">
```

Element / Class / ID Selectors

An element selector targets every instance of a tag; a class selector (prefixed with a dot) targets elements sharing that class; an ID selector (prefixed with #) targets the single element with that id.

SYNTAX

```
tagname { }  
.classname { }  
#idname { }
```

EXAMPLE

```
h2 { color: navy; }  
.card { padding: 16px; }  
#main-nav { background: #222; }
```

Descendant & Child Selectors

A descendant selector matches an element nested anywhere inside another; a child selector (>) matches only direct children.

SYNTAX

```
ancestor descendant { }  
parent > child { }
```

EXAMPLE

```
nav ul { list-style: none; }  
div > p { margin: 0; }
```

Pseudo-classes & Pseudo-elements

Pseudo-classes style an element based on its state (:hover, :focus, :nth-child); pseudo-elements style a specific part of an element (::before, ::after, ::first-letter).

SYNTAX

```
selector:pseudo-class { }  
selector::pseudo-element { }
```

EXAMPLE

```
a:hover { text-decoration: underline; }  
li:nth-child(odd) { background: #f2f2f2; }  
p::first-letter { font-size: 2em; }
```

Grouping & Combining Selectors

A comma applies the same declaration block to multiple selectors at once, avoiding repetition.

SYNTAX

```
selector1, selector2 { }
```

EXAMPLE

```
h1, h2, h3 {  
  font-family: 'Helvetica', sans-serif;  
}
```

8. The Box Model

Every HTML element is rendered as a rectangular box made of content, padding, border, and margin, from the inside out.

width / height

Sets the size of an element's content box. Values can be in px, %, em, rem, vw/vh, etc.

SYNTAX

```
selector { width: value; height: value; }
```

EXAMPLE

```
.box {  
  width: 300px;  
  height: 150px;  
}
```

padding

Adds space between an element's content and its border. A single value applies to all four sides; shorthand can supply 1, 2, 3, or 4 values.

SYNTAX

```
selector { padding: top right bottom left; }
```

EXAMPLE

```
.card {  
  padding: 16px;           /* all sides */  
  padding: 10px 20px;      /* vertical horizontal */  
}
```

border

Draws a line around an element's padding box. Shorthand combines width, style (solid, dashed, dotted), and color.

SYNTAX

```
selector { border: width style color; }
```

EXAMPLE

```
.card {  
  border: 2px solid #cccccc;  
  border-radius: 8px;  
}
```

margin

Adds space outside an element's border, separating it from neighboring elements. 'margin: 0 auto;' is a common trick to horizontally center a block element.

SYNTAX

```
selector { margin: top right bottom left; }
```

EXAMPLE

```
.container {  
  margin: 0 auto;  
  max-width: 960px;  
}
```

box-sizing

Controls whether width/height include padding and border. border-box (the common choice) keeps the declared width fixed even after adding padding and border.

SYNTAX

```
selector { box-sizing: content-box | border-box; }
```

EXAMPLE

```
* {  
  box-sizing: border-box;  
}
```

9. Typography & Colors

font-family / font-size / font-weight

Controls the typeface, size, and boldness of text. font-weight ranges from 100 (thin) to 900 (black); 'normal' and 'bold' are common keywords.

SYNTAX

```
selector {  
  font-family: value;  
  font-size: value;  
  font-weight: value;  
}
```

EXAMPLE

```
body {  
  font-family: 'Segoe UI', Arial, sans-serif;  
  font-size: 16px;  
  font-weight: 400;  
}
```

color / background-color

'color' sets text color; 'background-color' sets an element's background. Values can be named colors, hex codes, rgb()/rgba(), or hsl().

SYNTAX

```
selector { color: value; background-color: value; }
```

EXAMPLE

```
.alert {  
  color: #ffffff;  
  background-color: rgba(220, 53, 69, 0.9);  
}
```

text-align / text-decoration

Controls horizontal alignment of text and decorative lines such as underlines, often used to remove the default underline on links.

SYNTAX

```
selector {  
  text-align: left | right | center | justify;  
  text-decoration: none | underline | line-through;  
}
```

EXAMPLE

```
a {  
  text-decoration: none;  
}  
.title {  
  text-align: center;  
}
```

line-height / letter-spacing

line-height sets the vertical space between lines of text; letter-spacing adjusts the horizontal gap between characters.

SYNTAX

```
selector { line-height: value; letter-spacing: value; }
```

EXAMPLE

```
p {  
  line-height: 1.6;  
  letter-spacing: 0.5px;  
}
```

10. Flexbox Layout

Flexbox arranges items in a single row or column, distributing space and aligning content flexibly. Set `display: flex` on the parent container.

display: flex

Turns an element into a flex container, causing its direct children to become flex items laid out in a row by default.

SYNTAX

```
.container { display: flex; }
```

EXAMPLE

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
}
```

flex-direction

Sets the main axis along which flex items are placed — horizontally (row) or vertically (column).

SYNTAX

```
.container { flex-direction: row | row-reverse | column | column-reverse; }
```

EXAMPLE

```
.stack {  
  display: flex;  
  flex-direction: column;  
}
```

justify-content

Aligns flex items along the main axis, controlling how extra space is distributed between and around them.

SYNTAX

```
.container { justify-content: flex-start | center | space-between | space-around; }
```

EXAMPLE

```
.navbar {  
  display: flex;  
  justify-content: center;  
}
```

align-items

Aligns flex items along the cross axis (perpendicular to flex-direction).

SYNTAX

```
.container { align-items: flex-start | center | flex-end | stretch; }
```

EXAMPLE

```
.card-row {  
  display: flex;  
  align-items: center;  
}
```

gap

Adds consistent spacing between flex (or grid) items without needing margins on each one.

SYNTAX

```
.container { gap: value; }
```

EXAMPLE

```
.button-group {  
  display: flex;  
  gap: 12px;  
}
```

flex

Shorthand controlling how a flex item grows or shrinks relative to its siblings. 'flex: 1;' makes an item expand to fill available space equally.

SYNTAX

```
.item { flex: grow shrink basis; }
```

EXAMPLE

```
.sidebar { flex: 0 0 200px; }  
.main-content { flex: 1; }
```

11. CSS Grid Layout

Grid arranges items into two-dimensional rows and columns, ideal for full page layouts. Set `display: grid` on the parent container.

display: grid

Turns an element into a grid container so its children can be placed into rows and columns.

SYNTAX

```
.container { display: grid; }
```

EXAMPLE

```
.layout {  
  display: grid;  
}
```

grid-template-columns / rows

Defines the number and size of columns/rows. The `fr` unit distributes remaining space proportionally; `repeat()` avoids writing the same value multiple times.

SYNTAX

```
.container {  
  grid-template-columns: value value ...;  
  grid-template-rows: value value ...;  
}
```

EXAMPLE

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 16px;  
}
```

grid-column / grid-row

Positions an individual grid item across specific column or row lines, letting it span multiple tracks.

SYNTAX

```
.item {  
  grid-column: start / end;  
  grid-row: start / end;  
}
```

EXAMPLE

```
.featured {  
  grid-column: 1 / 3;  
  grid-row: 1 / 2;  
}
```

grid-template-areas

Names regions of the grid as a visual ASCII map, then assigns each child to a named area with the `grid-area` property — a very readable way to build page layouts.

SYNTAX

```
.container {  
  grid-template-areas:  
    "header header"
```

```
    "sidebar main"  
    "footer footer";  
}
```

EXAMPLE

```
.page {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-areas:  
    "sidebar main";  
}  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }
```

12. Positioning, Display & Responsive Design

display

Controls how an element generates boxes for layout. 'none' removes the element from the page entirely (it takes up no space).

SYNTAX

```
selector { display: block | inline | inline-block | flex | grid | none; }
```

EXAMPLE

```
.hidden {  
  display: none;  
}  
.icon {  
  display: inline-block;  
}
```

position

Determines how an element is placed. 'relative' offsets it from its normal position; 'absolute' positions it relative to the nearest positioned ancestor; 'fixed' pins it to the viewport; 'sticky' toggles between relative and fixed based on scroll position.

SYNTAX

```
selector { position: static | relative | absolute | fixed | sticky; }
```

EXAMPLE

```
.modal {  
  position: fixed;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

z-index

Controls stacking order among overlapping positioned elements — higher values appear on top of lower ones.

SYNTAX

```
selector { z-index: number; }
```

EXAMPLE

```
.dropdown {  
  position: absolute;  
  z-index: 100;  
}
```

@media (responsive breakpoints)

Applies a block of CSS rules only when the viewport matches a given condition, the foundation of responsive, mobile-friendly design.

SYNTAX

```
@media (max-width: value) {  
  selector { property: value; }  
}
```

EXAMPLE

```
@media (max-width: 768px) {  
  .sidebar {  
    display: none;  
  }  
  .container {  
    grid-template-columns: 1fr;  
  }  
}
```

CSS Variables

Custom properties store reusable values (colors, spacing) at a scope, commonly :root for global use, then reference them anywhere with var().

SYNTAX

```
:root { --name: value; }  
selector { property: var(--name); }
```

EXAMPLE

```
:root {  
  --primary-color: #c0392b;  
  --spacing: 16px;  
}  
button {  
  background-color: var(--primary-color);  
  padding: var(--spacing);  
}
```

End of reference guide. This document is intended as a teaching aid — encourage students to build a small practice page and experiment with each tag and property directly in the browser's developer tools.